

1.2 Technical AI/Machine Learning Recap

Dr. Max Lamparth
Stanford Intelligent Systems Laboratory
Hoover Institution

CS120 Stanford | 09/25/2025





Logistics

Zoom Recording Access

Class Size/Auditing/Enrollment Issues?

→ Please enroll in canvas in any situation

Anonymous Feedback Form + OH Link on Webpage

First OH next Wednesday

Gradescope/Ed

First assignment should be online now, due 10/07



What are your questions?



Slido open until end of class



What Is The Difference Between AI and ML?



Artificial Intelligence (AI)

is field of computer science

focused on **creating systems** capable of **performing tasks** that normally **require human intelligence**.

AI aims to **mimic or replicate** human **intelligence**

can encompass everything from simple algorithms that solve specific problems to **complex systems** that **learn and evolve**.

Artificial Intelligence

https://en.wikipedia.org/wiki/Machine_learning



What Is The Difference Between AI and ML?



Artificial Intelligence (AI)

is field of computer science

focused on **creating systems** capable of **performing tasks** that normally **require human intelligence**.

AI aims to **mimic or replicate** human **intelligence**

can encompass everything from simple algorithms that solve specific problems to **complex systems** that **learn and evolve**.

Machine Learning (ML)

is a field of study in artificial intelligence

concerned with the **development** and **study** of **statistical algorithms**

that can **learn from data** and **generalize to unseen data**, and thus perform tasks without explicit instructions.

Artificial Intelligence

Machine Learning

Deep Learning

https://en.wikipedia.org/wiki/Machine_learning



Machine Learning: Linear Regression



Arguably the simplest ML example: Linear regression

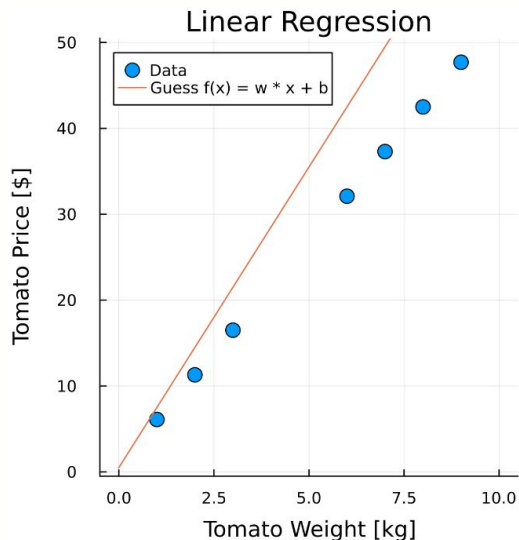
Goal: Take n-dim vector x as input, predict scalar value y

Assumption: Assume a linear relation between x and y

Model: Parameters w, b

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

How to set parameters, for a data set of (x_i, y_i) ?



Machine Learning: Linear Regression



Arguably the simplest ML example: Linear regression

Goal: Take n-dim vector x as input, predict scalar value y

Assumption: Assume a linear relation between x and y

Model: Parameters w, b

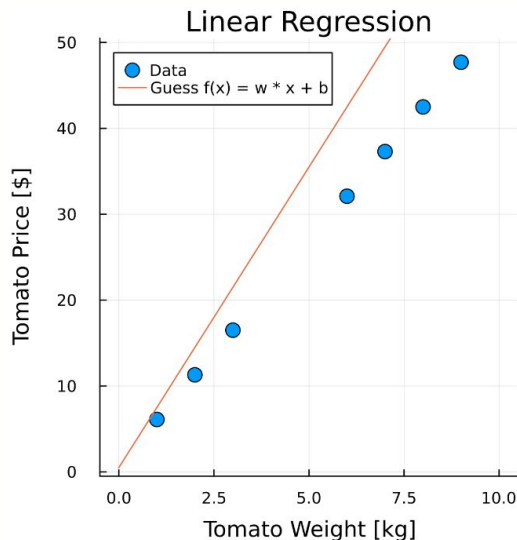
$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

How to set parameters, for a data set of (x_i, y_i) ?

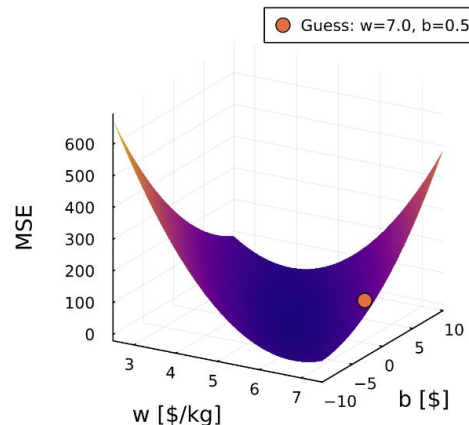
Need a measure: Choose mean-squared-error (MSE)

$$\text{MSE}_{\text{test}} = \frac{1}{m} \sum_i (\hat{\mathbf{y}}^{(\text{test})}_i - \mathbf{y}^{(\text{test})}_i)^2$$

→ Minimize MSE to find optimal w, b (grad MSE = 0)



Mean-Squared-Error 3D

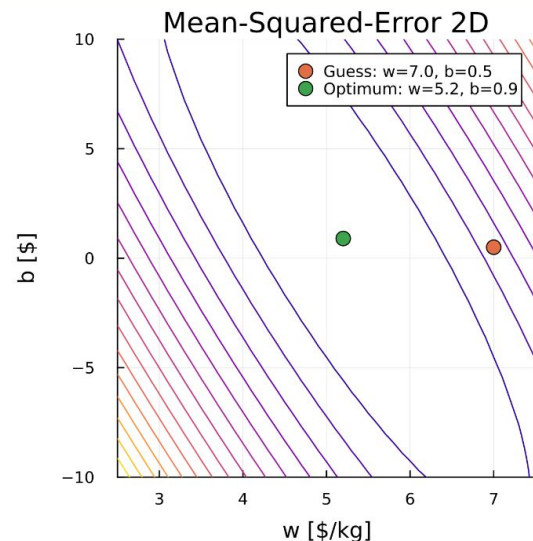
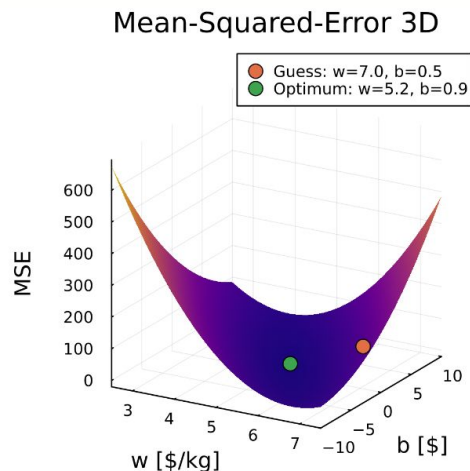
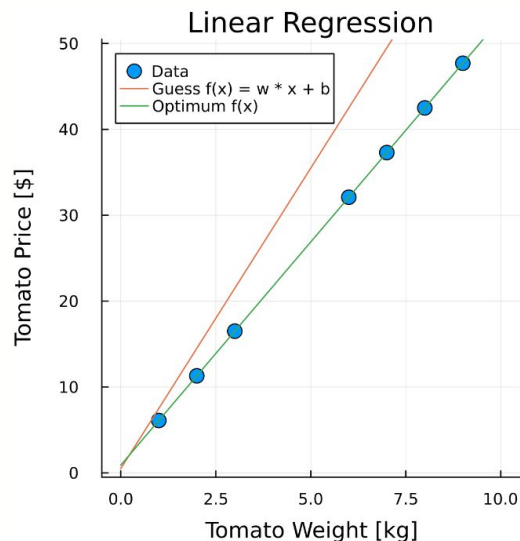


$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

$$\Rightarrow \mathbf{w} = \left(\mathbf{X}^{(\text{train})\top} \mathbf{X}^{(\text{train})} \right)^{-1} \mathbf{X}^{(\text{train})\top} \mathbf{y}^{(\text{train})}$$



Machine Learning: Linear Regression

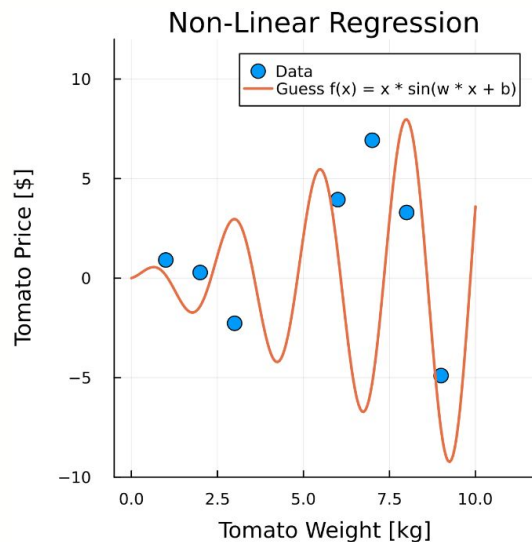


Assumes: Training and any new data is **independent and identically distributed (iid)**

In general: performance measure, like MSE, is called **loss function**



Machine Learning: Non-Linear Regression

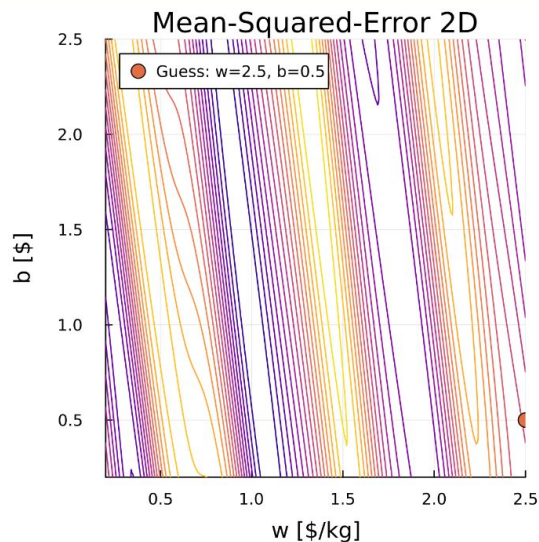
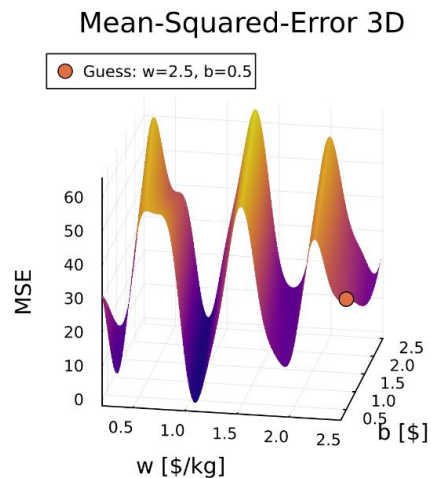
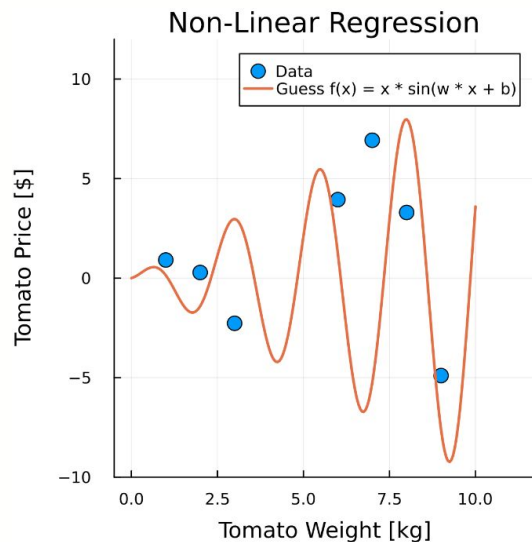


What do do if $\nabla_w \text{MSE}_{\text{train}} = 0$ does not work?

→ Still two parameters



Machine Learning: Non-Linear Regression



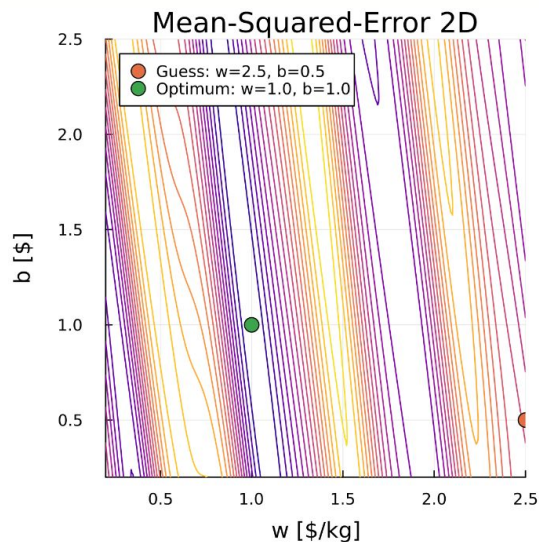
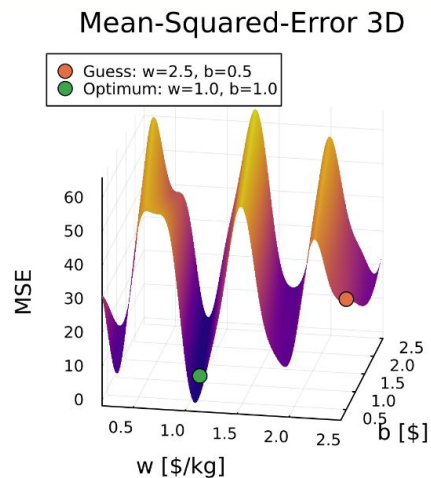
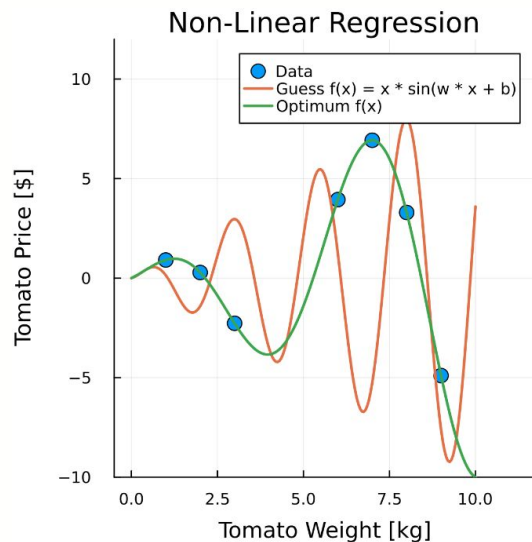
What do do if $\nabla_w \text{MSE}_{\text{train}} = 0$ does not work?

→ Still two parameters

→ Multiple minima



Machine Learning: Non-Linear Regression



What do do if $\nabla_w \text{MSE}_{\text{train}} = 0$ does not work?

→ Still two parameters

→ Multiple minima





What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

Negative conditional log-likelihood
(performance measure, just like MSE)

$\boldsymbol{\theta}$





What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:

Expected value if $\mathbf{x}, y$ sample from data distribution

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} \underbrace{L(\mathbf{x}, y, \boldsymbol{\theta})}_{\text{Per data example loss function}} = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

Negative conditional log-likelihood
(performance measure, just like MSE)

Per data example loss function:

$$L(\mathbf{x}, y, \boldsymbol{\theta}) = -\log p(y \mid \mathbf{x}; \boldsymbol{\theta})$$



Gradient Descent



What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:

Expected value if $\mathbf{x}, y$ sample from data distribution

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} \underbrace{L(\mathbf{x}, y, \boldsymbol{\theta})}_{\text{Per data example loss function}} = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

Negative conditional log-likelihood
(performance measure, just like MSE)

Per data example loss function:

$$L(\mathbf{x}, y, \boldsymbol{\theta}) = -\log p(y \mid \mathbf{x}; \boldsymbol{\theta})$$

Given $\mathbf{x}$ and current set of parameters $\boldsymbol{\theta}$,
what is the probability of y ?



Gradient Descent



What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:

Expected value if $\mathbf{x}, y$ sample from data distribution

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} \underbrace{L(\mathbf{x}, y, \boldsymbol{\theta})}_{\text{Per data example loss function:}} = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

Number of training data points m

Negative conditional log-likelihood
(performance measure, just like MSE)

Per data example loss function:

$$L(\mathbf{x}, y, \boldsymbol{\theta}) = -\log p(y \mid \mathbf{x}; \boldsymbol{\theta})$$

Given $\mathbf{x}$ and current set of parameters $\boldsymbol{\theta}$,
what is the probability of y ?



Gradient Descent

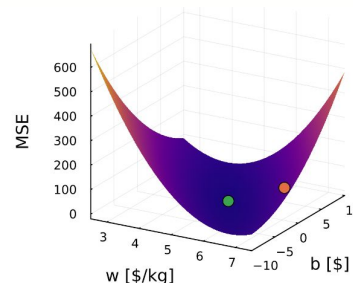


What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$



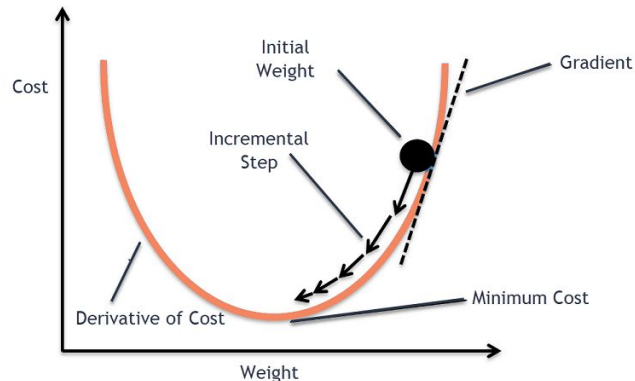
Gradient Descent



What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:



$$J(\theta) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \theta) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

Still need a gradient:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

→ Update parameters θ in small steps along gradient
in units of learning rate ϵ :

$$\theta \leftarrow \theta - \epsilon \nabla_{\theta} J(\theta)$$

Goodfellow, I., Bengio, Y., Courville, A. Deep Learning. MIT Press (2016).

<https://www.analyticsvidhya.com/blog/2020/10/how-does-the-gradient-descent-algorithm-work-in-machine-learning/>





What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:

Hyperparameters: Not trainable, external configuration variables to manage training.

“Machine Learning [...] development and study of statistical algorithms that can learn from and generalize to unseen data”

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

Still need a gradient:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

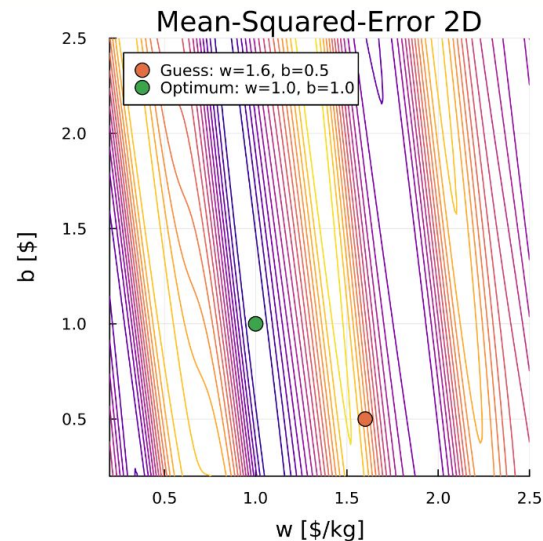
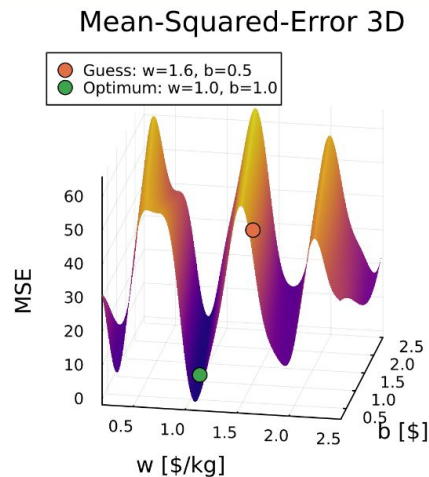
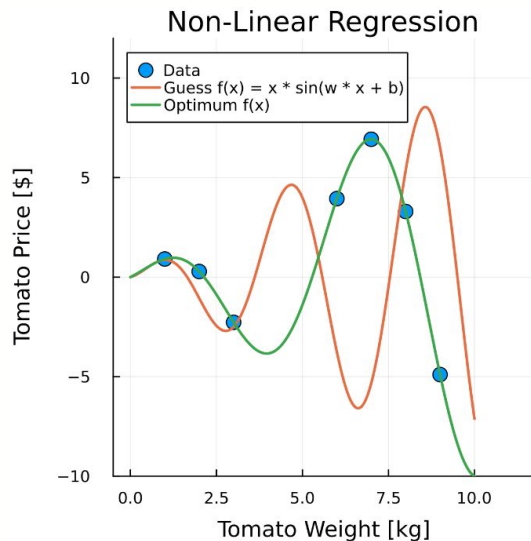
→ Update parameters $\boldsymbol{\theta}$ in small steps along gradient
in units of **learning rate** ϵ :

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

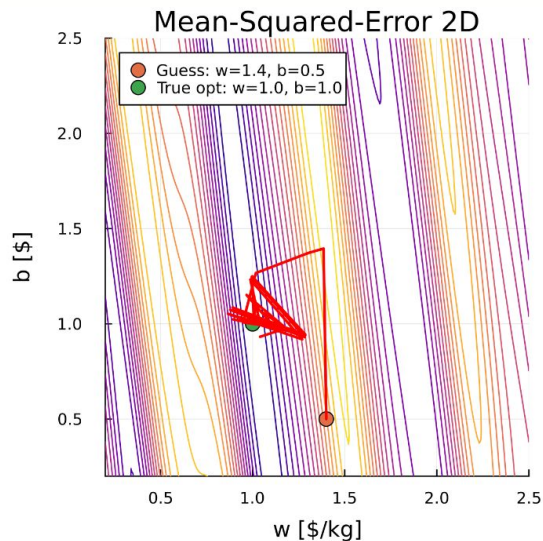
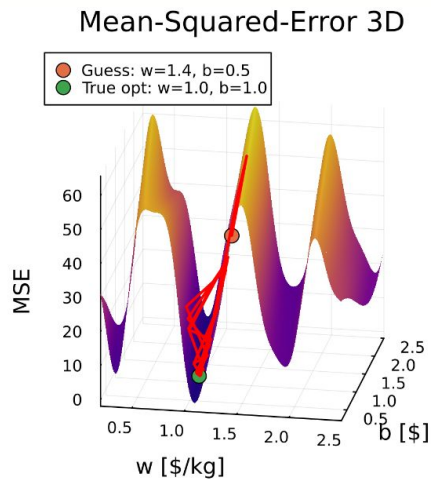
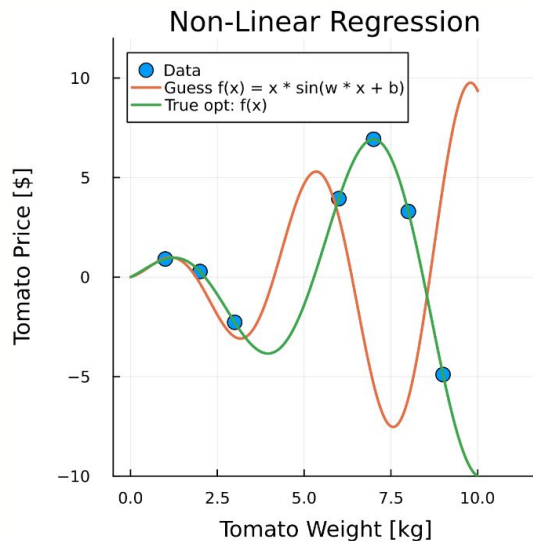
Goodfellow, I., Bengio, Y., Courville, A. Deep Learning. MIT Press (2016).



Gradient Descent



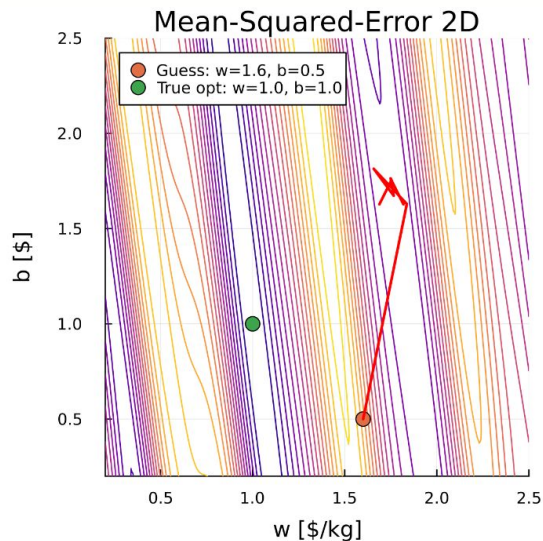
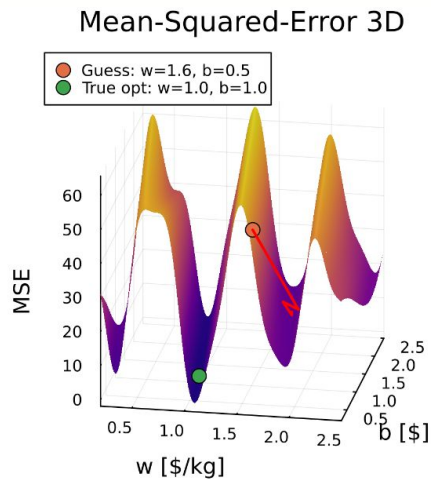
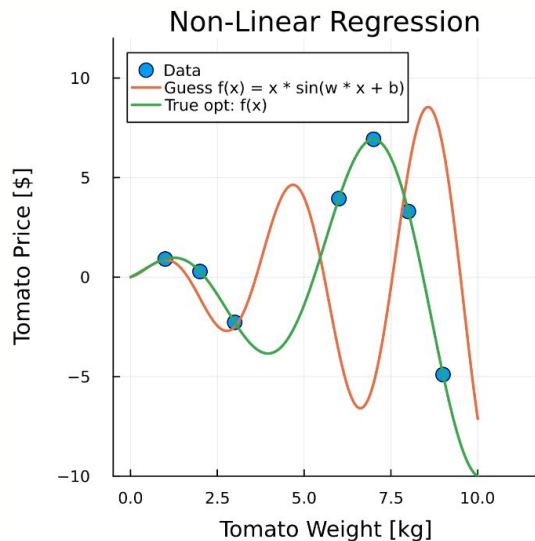
Gradient Descent



GD does stable gradient estimation
→ Can find global minima!



Gradient Descent



GD convergence depends on starting parameter

→ Can be “stuck” in local minima



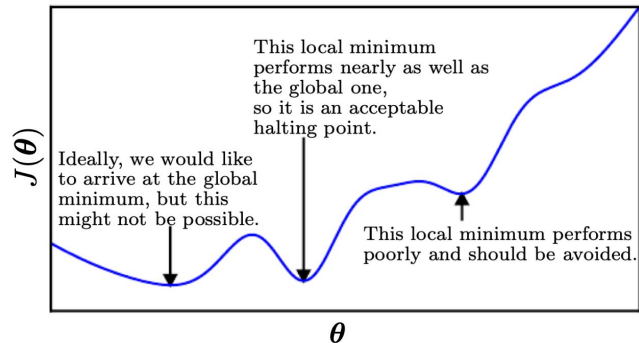
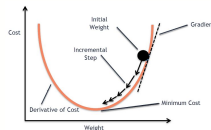
Gradient Descent



What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:



$$J(\theta) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \theta) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

Still need a gradient:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

→ Update parameters θ in small steps along gradient
in units of learning rate ϵ :

$$\theta \leftarrow \theta - \epsilon \nabla_{\theta} J(\theta)$$

→ Calculates exact gradient using **full data set**

→ **Converges locally**, global convergence not given

Goodfellow, I., Bengio, Y., Courville, A. Deep Learning. MIT Press (2016).





What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

Still need a gradient:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta}) \quad \rightarrow \quad \mathbf{g} = \frac{1}{m'} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^{m'} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

→ Update parameters $\boldsymbol{\theta}$ in small steps along gradient
in units of learning rate ϵ :

Approximate gradient with subset of data!

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \mathbf{g},$$

Goodfellow, I., Bengio, Y., Courville, A. Deep Learning. MIT Press (2016).





What if there are multiple minima with zero gradient?

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0$$

View problem as more general statistics problem:

Algorithm 8.1 Stochastic gradient descent (SGD) update at training iteration k

Require: Learning rate ϵ_k .

Require: Initial parameter θ

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow +\frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Apply update: $\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$

end while

$$J(\theta) = \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} L(\mathbf{x}, \mathbf{y}, \theta) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}, \theta)$$

Still need a gradient:

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}, \theta) \quad \longrightarrow \quad \mathbf{g} = \frac{1}{m'} \nabla_{\theta} \sum_{i=1}^{m'} L(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}, \theta)$$

→ Update parameters θ in small steps along gradient
in units of learning rate ϵ :

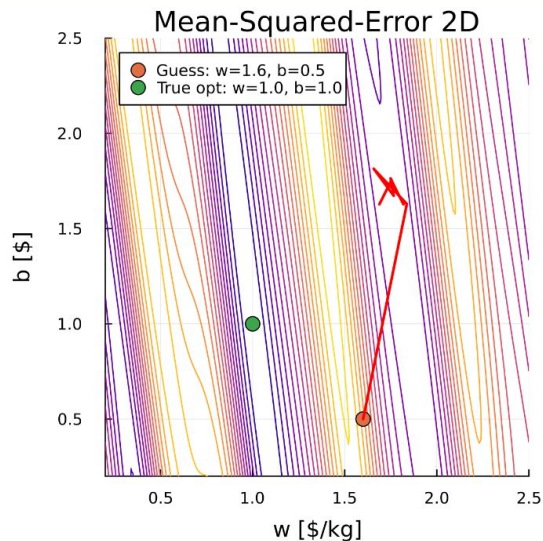
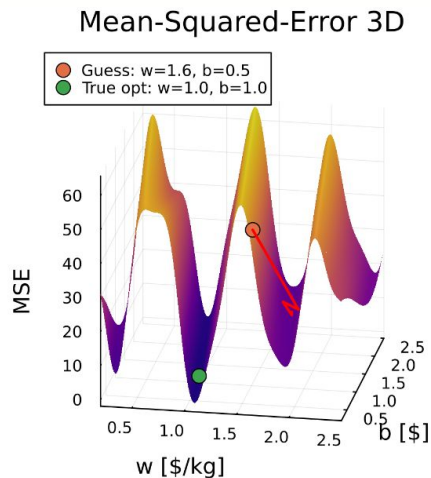
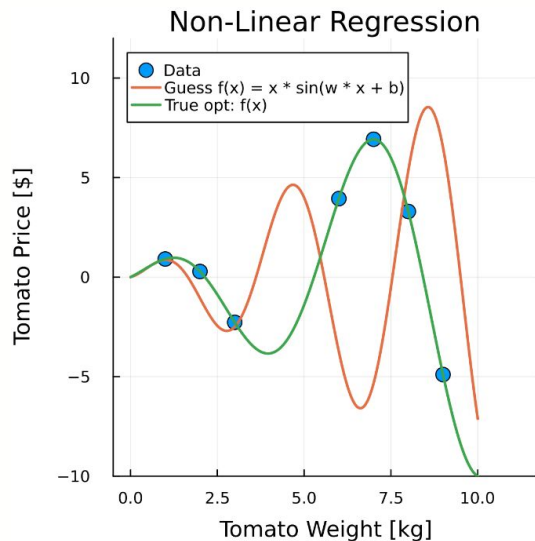
$$\theta \leftarrow \theta - \epsilon \mathbf{g},$$

Approximate gradient with subset of data!

Goodfellow, I., Bengio, Y., Courville, A. Deep Learning. MIT Press (2016).



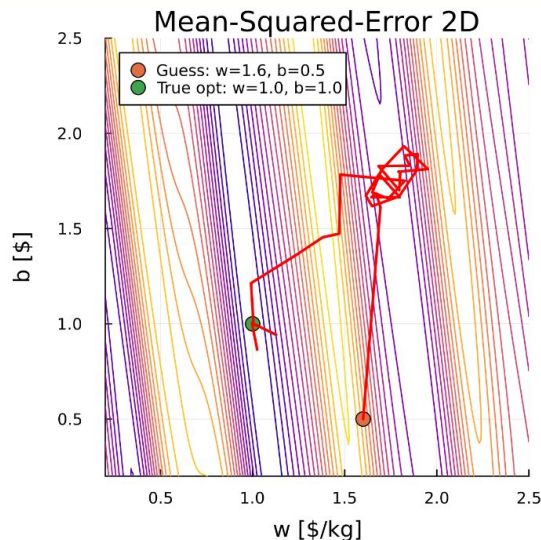
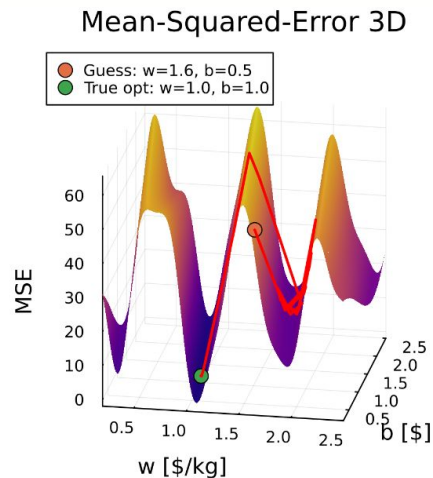
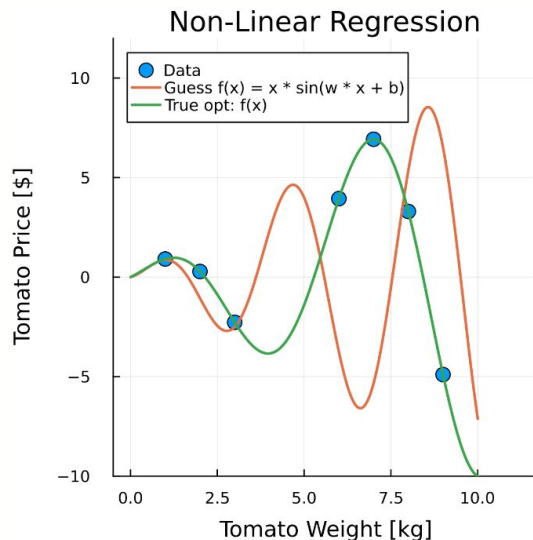
Gradient Descent



GD convergence depends on starting parameter
→ Can be “stuck” in local minima



Stochastic Gradient Descent



SGD has noisier parameter updates

→ Can also potentially escape local minima

(But can also fall into same local minima)

Follow-up topics:

- Bias variance trade-off, cross-validation
- Automatic differentiation (how to get gradients?)
- Other optimizers, e.g., ADAM, RMS-Prop, ...
(Better convergence than SGD?)





What function should I use? A flexible one!

→ More parameters = more possible solutions we can use?





What function should I use? A flexible one!

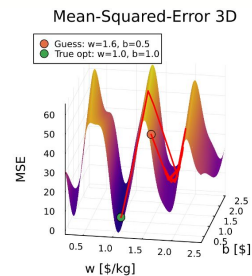
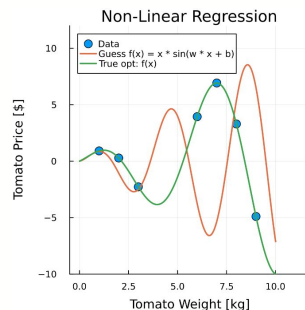
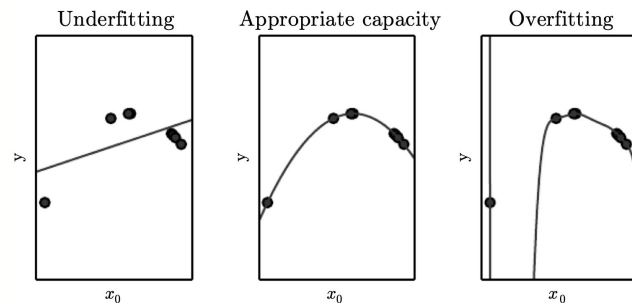
→ More parameters = more possible solutions we can use?

Not quite!

→ Also depends on available class of functions in possible solutions

E.g., infinite amount of linear functions, but only works where relationship between inputs and outputs is linear

- Often want to learn non-linear function
- Need flexible function and need to deal with bumpy loss landscape





What function should I use? A flexible one!

→ More parameters = more possible solutions we can use?

Turn linear...

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{x}^\top \mathbf{w} + b.$$





Neural Networks

What function should I use? A flexible one!

→ More parameters = more possible solutions we can use?

Turn linear...

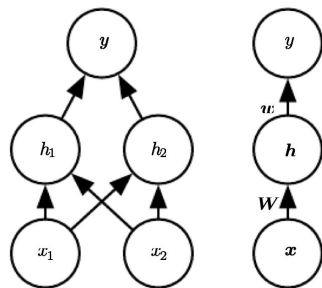
$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{x}^\top \mathbf{w} + b.$$

... into non-linear by applying a non-linear function (“**activation function**”)

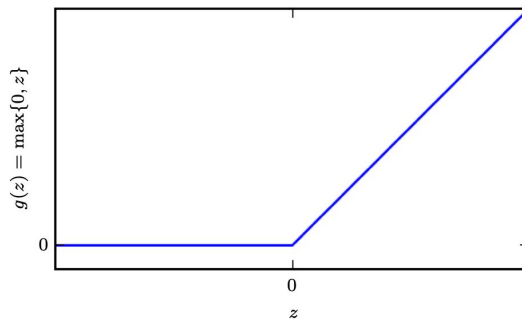
$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = \mathbf{w}^\top \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\} + b.$$

General notation for an input $\mathbf{x}$ and output hidden state $\mathbf{h}$:

$$\mathbf{h} = g(\mathbf{W}^\top \mathbf{x} + \mathbf{b}).$$



Rectified Linear Unit (ReLU)



Neural Networks



What function should I use? A flexible one!

→ More parameters = more possible solutions we can use?

Turn linear...

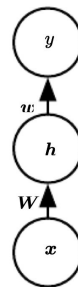
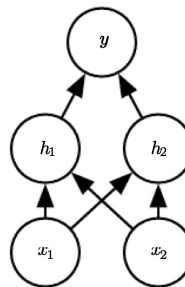
$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{x}^\top \mathbf{w} + b.$$

... into non-linear by applying a non-linear function (“**activation function**”)

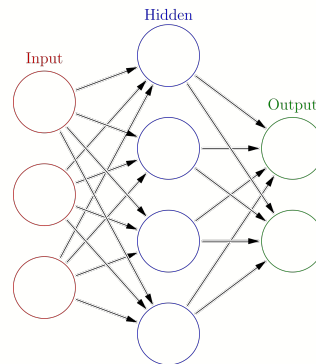
$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = \mathbf{w}^\top \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\} + b.$$

General notation for an input $\mathbf{x}$ and output hidden state $\mathbf{h}$:

$$\mathbf{h} = g(\mathbf{W}^\top \mathbf{x} + \mathbf{b}).$$



$$\mathbf{h}^{(2)} = g^{(2)} \left(\mathbf{W}^{(2)\top} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \right),$$
$$\mathbf{h}^{(1)} = g^{(1)} \left(\mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)} \right),$$

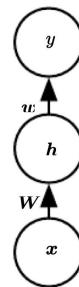
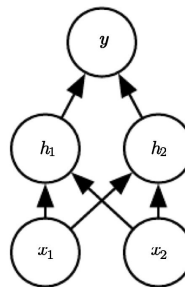


Neural Networks



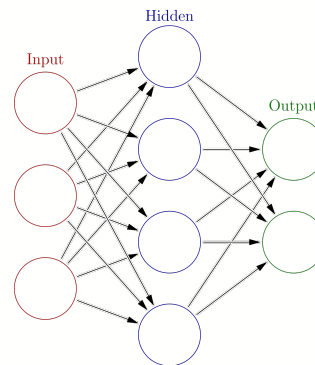
What function should I use? A flexible one!

→ More parameters = more possible solutions we can use?



$$\mathbf{h}^{(2)} = g^{(2)} \left(\mathbf{W}^{(2)\top} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \right),$$

$$\mathbf{h}^{(1)} = g^{(1)} \left(\mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)} \right),$$



Neural Networks



What function should I use? A flexible one!

→ More parameters = more possible solutions we can use!

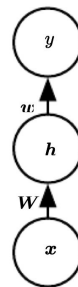
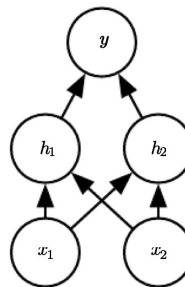
Universal function approximation theorem

A **neural network** with **one hidden layer** with a **non-linear activation function** (multi-layer perceptron, MLP) will be able to **represent any function** we want to learn, if hidden layer large enough

No guarantee for training algorithm to find it, though

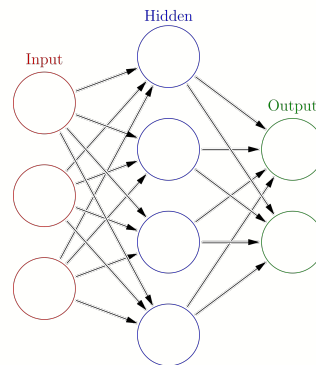
Proven for most relevant activation functions

Target function requirements: Continuous, needs to be Borel measurable, ...



$$\mathbf{h}^{(2)} = g^{(2)} \left(\mathbf{W}^{(2)\top} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \right),$$

$$\mathbf{h}^{(1)} = g^{(1)} \left(\mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)} \right),$$



Deep Neural Networks



What function should I use? A flexible one!

→ More parameters = more possible solutions we can use!

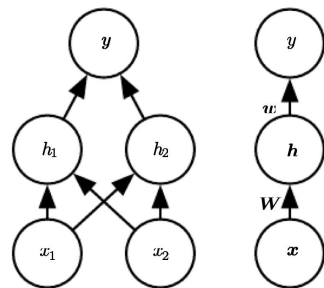
Universal function approximation theorem

A **neural network** with **one hidden layer** with a **non-linear activation function** (multi-layer perceptron, MLP) will be able to **represent any function** we want to learn, if hidden layer large enough

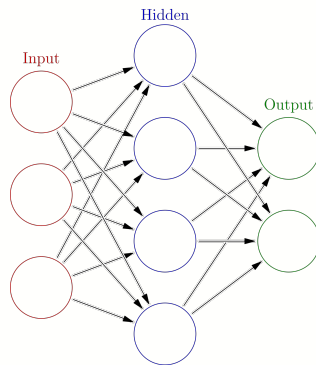
No guarantee for training algorithm to find it, though

Proven for most relevant activation functions

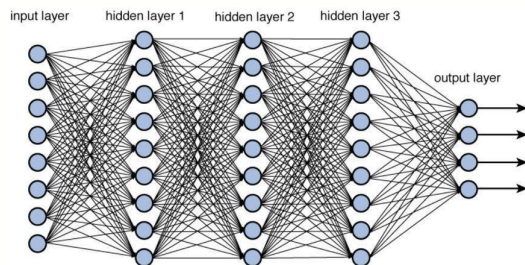
Target function requirements: Continuous, needs to be Borel measurable, ...



$$\mathbf{h}^{(2)} = g^{(2)} \left(\mathbf{W}^{(2)\top} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \right),$$
$$\mathbf{h}^{(1)} = g^{(1)} \left(\mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)} \right),$$



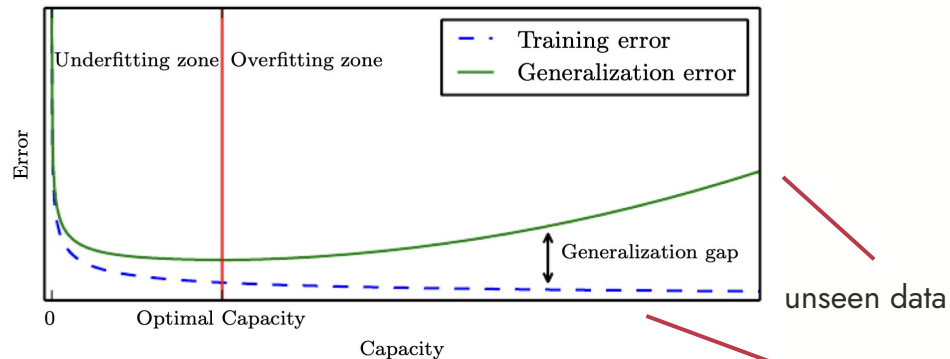
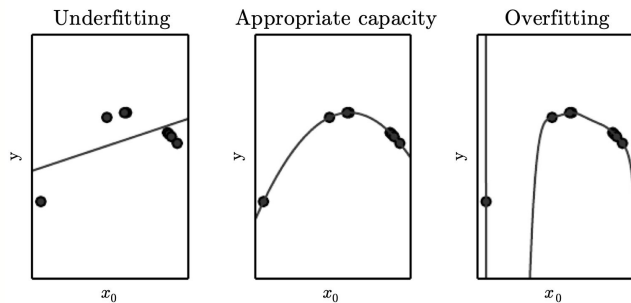
We need to go deeper!



Goodfellow, I., Bengio, Y., Courville, A. Deep Learning. MIT Press (2016).
[https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning))
<https://towardsdatascience.com/training-deep-neural-networks-9fdb1964b964>



Why Does Over-parameterization Work (in practice)?



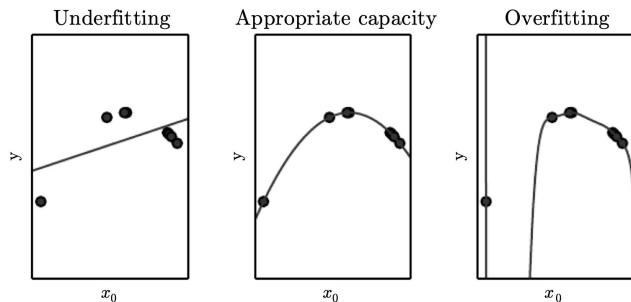
Performance measure (Loss) on training (seen) data

→ Problem that happens with too many parameters

Goodfellow, I., Bengio, Y., Courville, A. Deep Learning. MIT Press (2016).



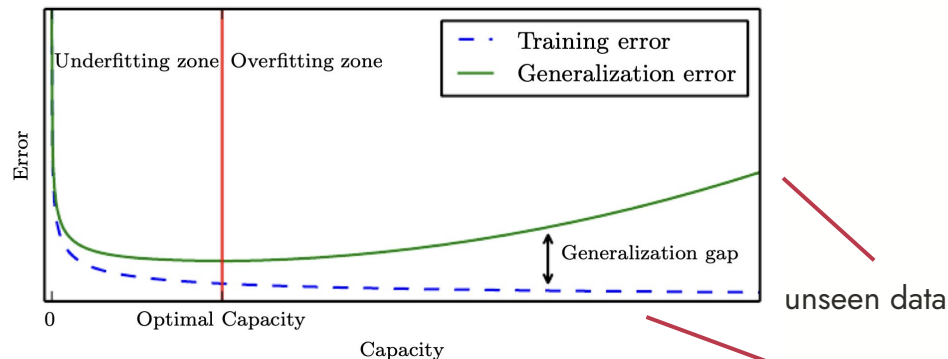
Why Does Over-parameterization Work (in practice)?



When training a model, we assume that each data point is independent of other data points and all data points follow the same underlying distribution (**Independent and Identically Distributed**, IID assumption)

Required to generalize from training to unseen data, under the expectation that the unseen data has same distribution.

But, in real-world applications, often violated due to, e.g., correlations, distribution changes over time, distribution shift (**out-of-distribution**, OOD performance)



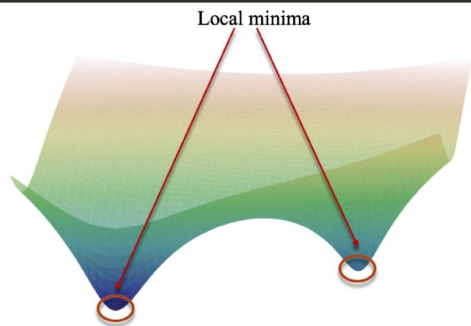
Performance measure (Loss) on training (seen) data

→ Problem that happens with too many parameters

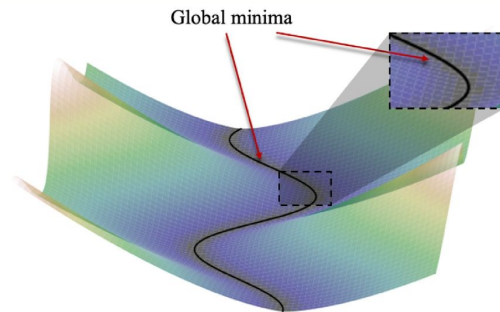
Goodfellow, I., Bengio, Y., Courville, A. Deep Learning. MIT Press (2016).



Why Does Over-parameterization Work (in practice)?



(a) Loss landscape of under-parameterized models



(b) Loss landscape of over-parameterized models

“Loss landscape of an over-parameterized system is generally not convex in any neighborhood of any global minimizer. [...] This is different from the case of under-parameterized systems, where the loss landscape is globally not convex, but still typically locally convex [...]”

“This condition ensures existence of solutions and convergence of GD and SGD [...] we show that sufficiently wide neural networks satisfy the PL* condition around their initialization point, thus guaranteeing convergence”.

“Convexity is not the right framework for analysis of over-parameterized systems, even locally.”

Follow-up topic: Regularization



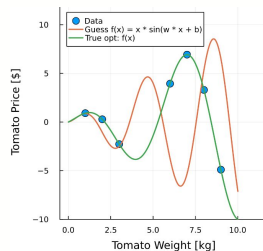
Intermediate Summary



What do we want to do?

Approximate functions given data

Create a learning algorithm that finds function approximation



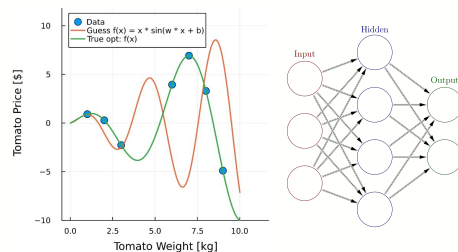
Intermediate Summary



What do we want to do?

Approximate functions given data

Create a learning algorithm that finds function approximation



What do we use as starting function?

A (deep) neural network

Why?

Universal function approximation theorem

(It can theoretically represent any function we need)



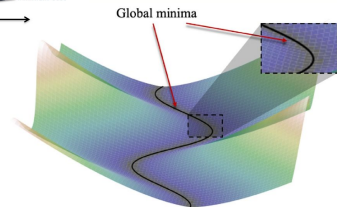
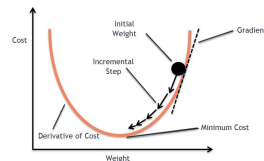
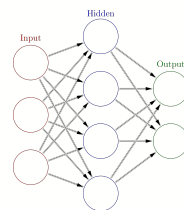
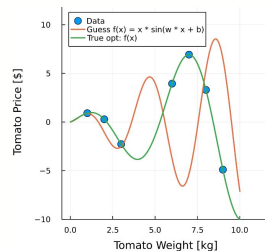
Intermediate Summary



What do we want to do?

Approximate functions given data

Create a learning algorithm that finds function approximation



What do we use as starting function?

A (deep) neural network

Why?

Universal function approximation theorem

(It can theoretically represent any function we need)

How do we create the learning algorithm?

Use SGD to incrementally tune parameters of neural network

Why?

“Noisy” updates from SGD require less data at once and can escape local minima

With vastly over-parameterized models, loss-landscape is easier for convergence





B R E A K (Questions?)

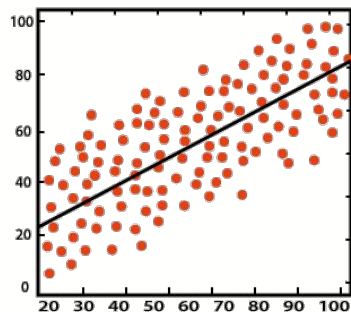


Regression vs. Classification



Regression

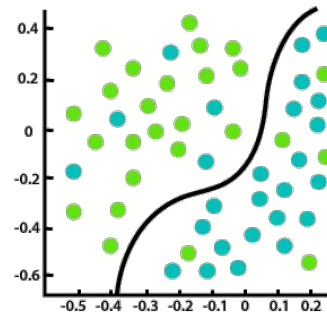
Predicting a continuous outcome or value based on one or more input variables.



Regression

Classification

Predicting which category or class an observation belongs to, based on one or more input variables. The output is discrete and typically involves categorizing into classes.



Classification

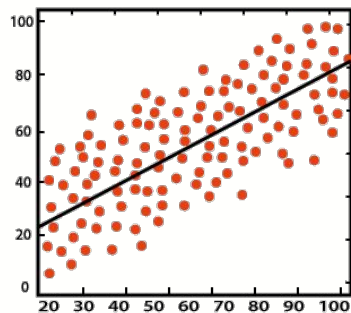


Regression vs. Classification



Regression

Predicting a continuous outcome or value based on one or more input variables.



Regression

Different output structure, e.g., **probabilities** (Kolmogorov axioms)
→ Different last layer/activation function at the end

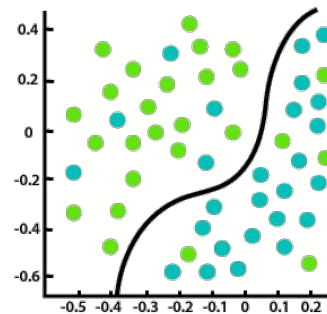
$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Different loss functions, e.g., **mean-square error** vs **cross-entropy loss**

$$\text{MSE}_{\text{test}} = \frac{1}{m} \sum_i (\hat{\mathbf{y}}^{(\text{test})} - \mathbf{y}^{(\text{test})})_i^2$$

Classification

Predicting which category or class an observation belongs to, based on one or more input variables. The output is discrete and typically involves categorizing into classes.



Classification

$$H(p, q) = - \sum_{x \in \text{classes}} p(x) \log q(x)$$

True probability distribution (one-shot)

Your model's predicted probability distribution

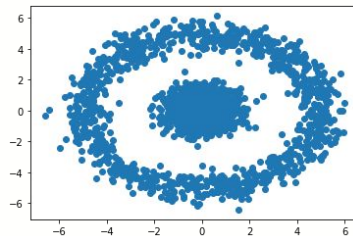


Latent Space Representations



What do machine learning models actually learn?

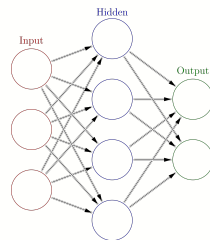
A **mapping** between input and outputs



Latent Space Representations

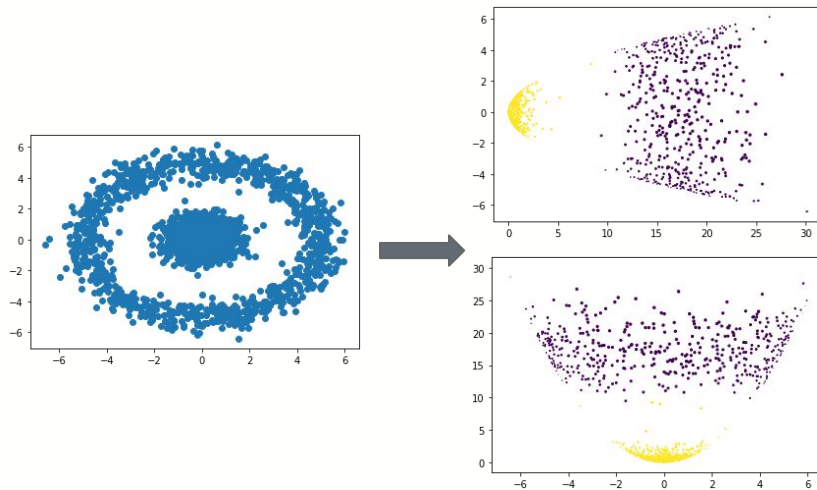
What do machine learning models actually learn?

A **mapping** between input and outputs



What if last layer is linear?

→ Learned **representations** must be linearly separable/usable

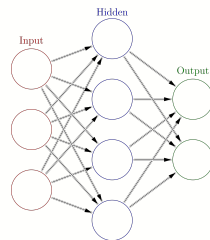


Latent Space Representations



What do machine learning models actually learn?

A **mapping** between input and outputs

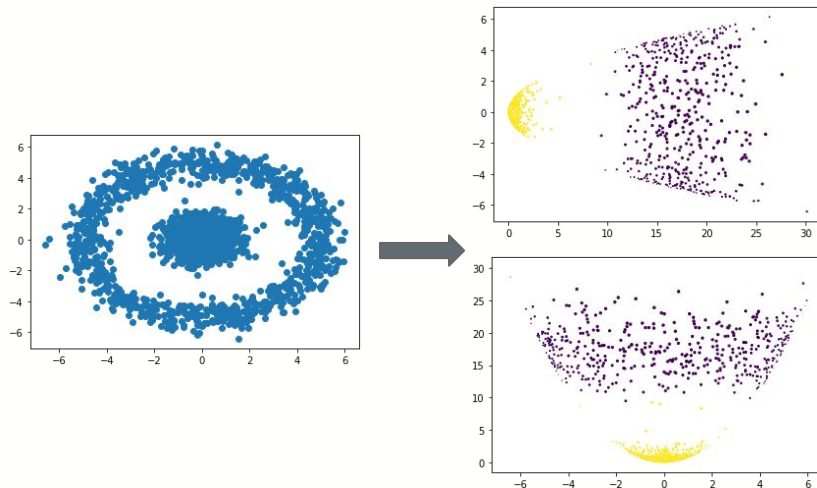


What if last layer is linear?

→ Learned **representations** must be linearly separable/usable

Representations = how data is mapped at different layers.
Each layer captures different features of the data, starting from very basic to more complex and abstract features at deeper layers. (**Hierarchical Feature Learning**)

Latent space refers to the representation space where deep learning models encode compressed, abstract features of the input data, capturing underlying patterns.



Autoencoders

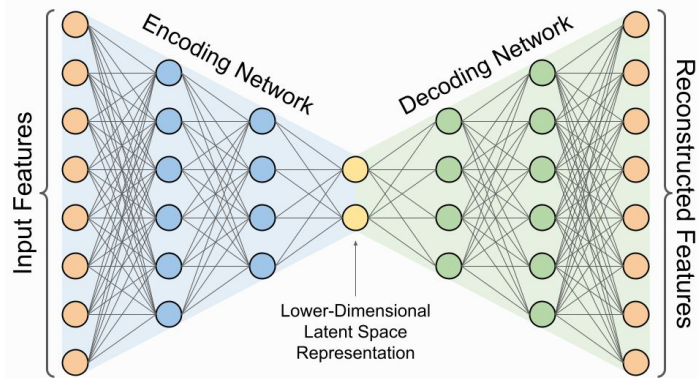


Goal: Learn a mapping that **recreates input data** through **bottleneck**

- **Autoencoders** must find good latent representations
- Can use **encoder** and **decoder separately** later!

Some applications:

data denoising, dimensionality reduction, feature learning, data-efficient pre-training neural networks, data compression, generative AI ...



Autoencoders

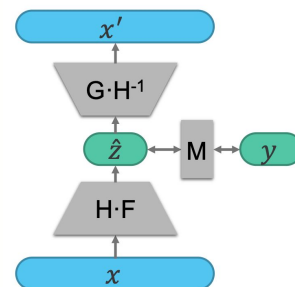
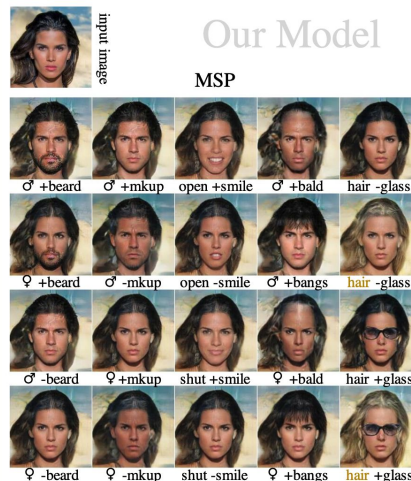
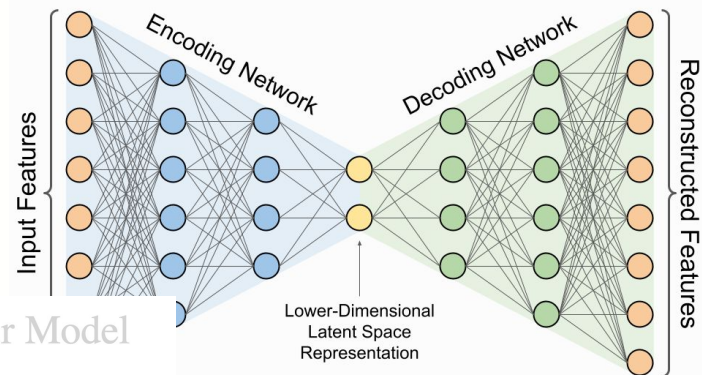
Goal: Learn a mapping that recreates input data through bottleneck

- Autoencoders must find good latent representations
- Can use encoder and decoder separately later!

Some applications:

data denoising, dimensionality reduction, feature learning, data-efficient pre-training neural networks, data compression, generative AI ...

Latent space **representations can be used to manipulate** generated data





Fine-tuning,

use an already trained (**pre-trained**) model
Train more on a smaller, specific dataset to adapt

→ Leverage better representations from initial training

Useful when

- you have a small dataset, training from scratch may lead to overfitting.
- New task is similar, i.e., if similar representations are actually useful
- Less computational power and time available
(Faster Convergence + Better Performance)



Pre-train model on
large data set

Fine-tune model on
smaller, task-specific
data set

Profit



Fine-Tuning



Fine-tuning,
use an already trained (pre-trained) model
Train more on a smaller, specific dataset to adapt
→ Leverage better representations from initial training

Useful when

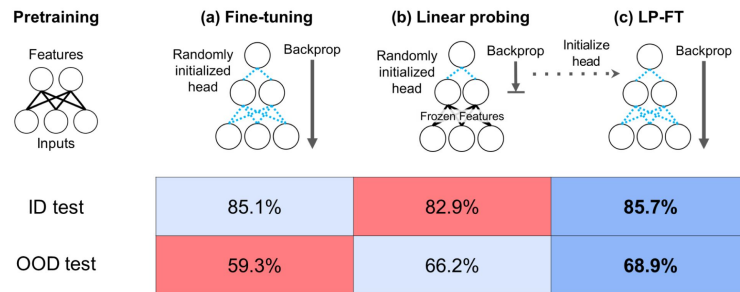
- you have a small dataset, training from scratch may lead to overfitting.
- New task is similar, i.e., if similar representations are actually useful
- Less computational power and time available
(Faster Convergence + Better Performance)

Full fine-tuning (updating all the model parameters) leads to better in-distribution performance

But, linear probing (updating only the last linear layer—the “head”) can lead to different OOD performance!

→ Full fine-tuning can distort the pre-trained representations

Best way: Do both, but start with linear probing



Average accuracies (10 distribution shifts)

Kumar et al. Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution.
Preprint at arxiv:2202.10054 (2022)





Fine-tuning,

use an already trained (pre-trained) model
Train more on a smaller, specific dataset to adapt

→ Leverage better representations from initial training

Useful when

- you have a small dataset, training from scratch may lead to overfitting.
- New task is similar, i.e., if similar representations are actually useful
- Less computational power and time available
(Faster Convergence + Better Performance)



Pre-train model on
large data set

Fine-tune last layer on
smaller, task-specific
data set

Fine-tune full model on
smaller, task-specific
data set

Profit



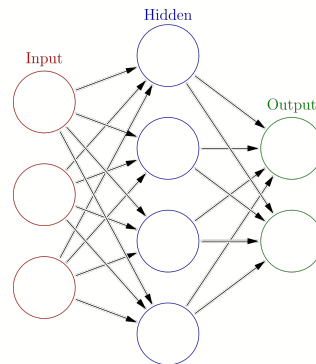
How To Handle Different Inputs?



Learning representations is great, but how do I work with

Numbers, vectors, matrices

→ Deep neural network, ...



How To Handle Different Inputs?



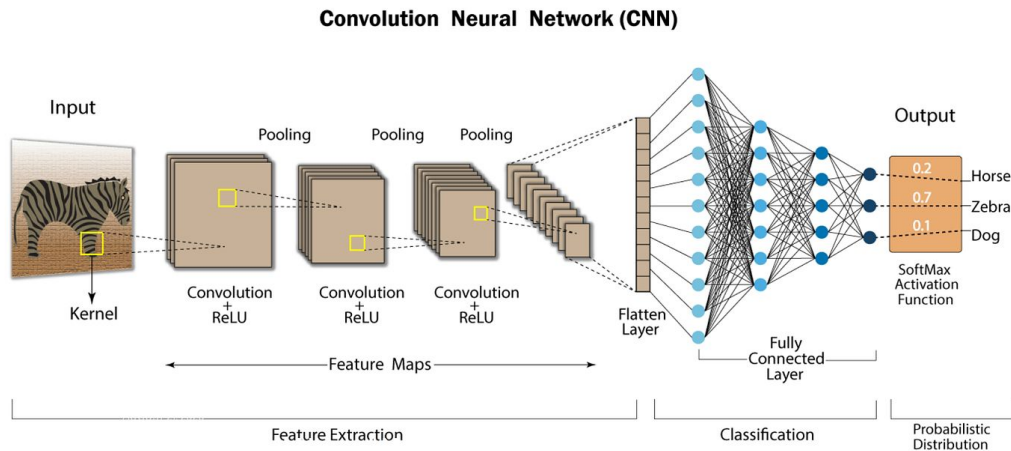
Learning representations is great, but how do I work with

Numbers, vectors, matrices

→ Deep neural network, ...

Images

→ **Convolutional neural network**, diffusion models, tokenization, ...



Use discrete convolution filter to re-use trainable weights over image pixels and (rgb) channels



How To Handle Different Inputs?

Learning representations is great, but how do I work with

Numbers, vectors, matrices

→ Deep neural network, ...

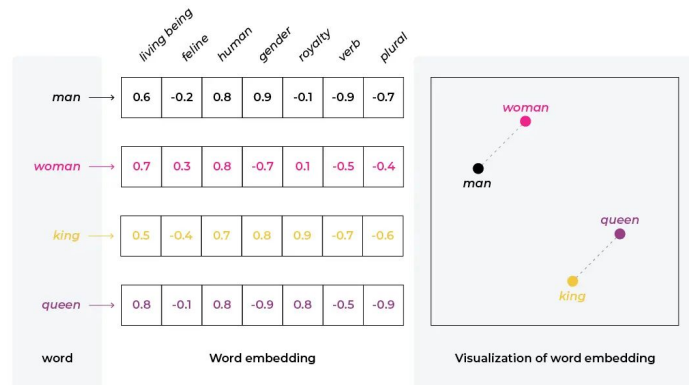
Images

→ Convolutional neural network, diffusion models, tokenization, ...

Text

→ **Embedding vectors** (word2vec/glove), tokenization, recurrent neural networks, transformer (attention + parallel sequence modelling), ...

Text + Images, Videos,



Learn a mapping of words (or sentences) into a high-dimensional vector space

→ Learns relationship and some semantics

Can be used to map back to words

<https://arize.com/blog-course/embeddings-meaning-examples-and-how-to-compute/>
Mikolov, T. et al. Efficient Estimation of Word Representations in Vector Space. Preprint at arxiv:1301.3781 (2013)

Pennington, R. et al. GloVe: Global Vectors for Word Representation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP) (2014)

How To Handle Different Inputs?

Learning representations is great, but how do I work with

Numbers, vectors, matrices

→ Deep neural network, ...

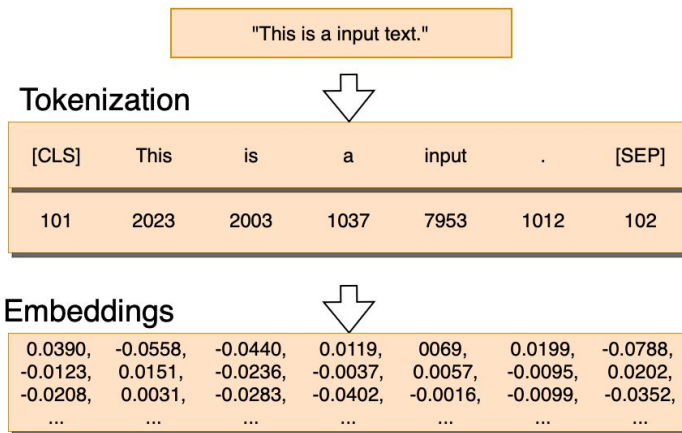
Images

→ Convolutional neural network, diffusion models, tokenization, ...

Text

→ Embedding vectors (word2vec/glove), **tokenization**, recurrent neural networks, transformer (attention + parallel sequence modelling), ...

Text + Images, Videos,



Find a function that separates a sequence of text (or an image) into small tokens (e.g., words or syllables)

Learn (separate) embedding for tokens

How To Handle Different Inputs?



Learning representations is great, but how do I work with

Numbers, vectors, matrices

→ Deep neural network, ...

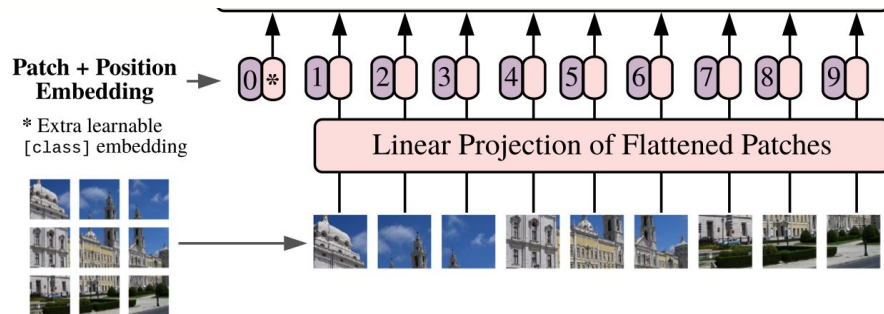
Images

→ Convolutional neural network, diffusion models, tokenization, ...

Text

→ Embedding vectors (word2vec/glove), **tokenization**, recurrent neural networks, transformer (attention + parallel sequence modelling), ...

Text + Images, Videos,



Find a function that separates a sequence of text (or an image) into small tokens (e.g., words or syllables)

Learn (separate) embedding for tokens



How To Handle Different Inputs?

Learning representations is great, but how do I work with

Numbers, vectors, matrices

→ Deep neural network, ...

Images

→ Convolutional neural network, diffusion models, tokenization, ...

Text

→ Embedding vectors (word2vec/glove), tokenization, **recurrent neural networks**, transformer (attention + parallel sequence modelling), ...

Text + Images, Videos,

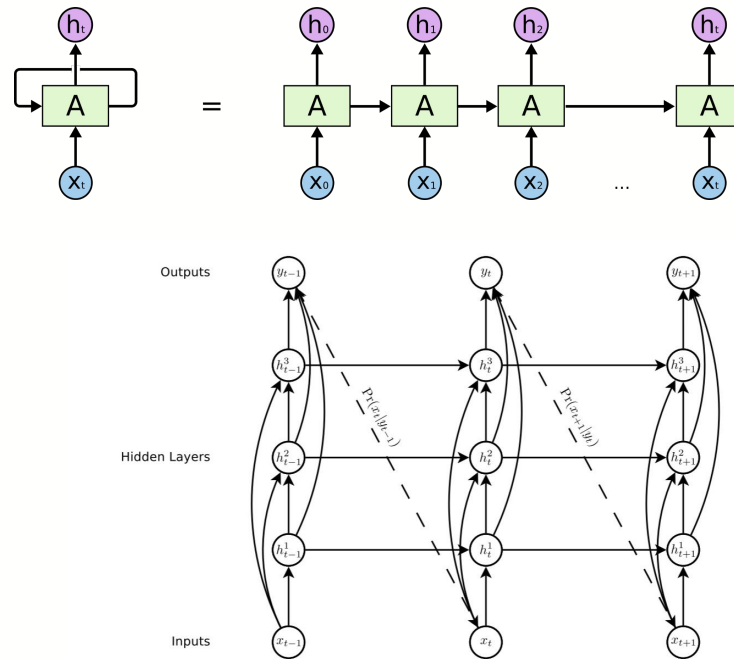


Figure 1: **Deep recurrent neural network prediction architecture.** The circles represent network layers, the solid lines represent weighted connections and the dashed lines represent predictions.

Better Training/Gradients?

Training deep neural networks can be difficult

→ Gradients can get small or very large when training deep models (**vanishing/exploding gradients problem**)

ResNet (2015) introduced skip-connections

- Allows models to learn identity function
- Mitigates vanishing gradient problem, as gradients can flow directly through the skip connections
- Architecture facilitates the reuse of earlier features

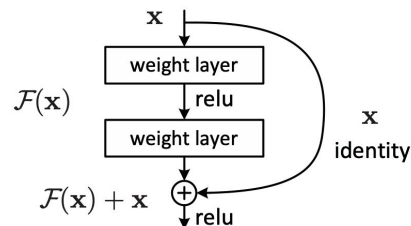
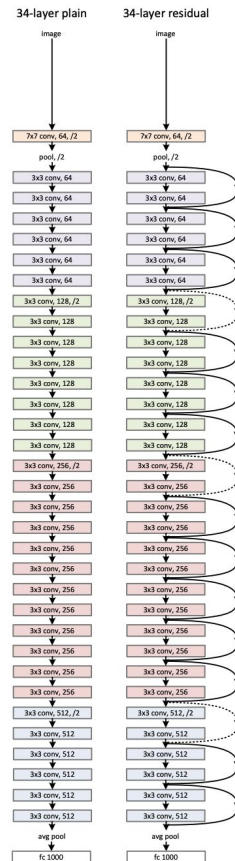


Figure 2. Residual learning: a building block.

Follow-up topics:

- Other architecture choices (Dropout, Batchnorm, Layernorm,...)
- Impact of weight initialization



Better Training/Gradients?



Training deep neural networks can be difficult

→ Gradients can get small or very large when training deep models (**vanishing/exploding gradients problem**)

ResNet (2015) introduced skip-connections

→ Allows models to learn identity function

→ Mitigates vanishing gradient problem, as gradients can flow directly through the skip connections

→ Architecture facilitates the reuse of earlier features

Can also be viewed as continuous stream ("**residual stream**") that is repeatedly modified by layer modules

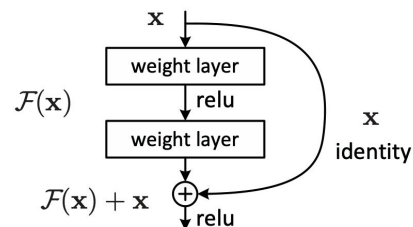
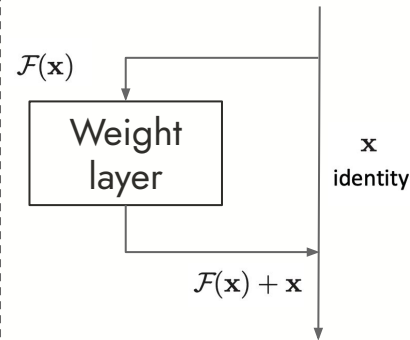


Figure 2. Residual learning: a building block.

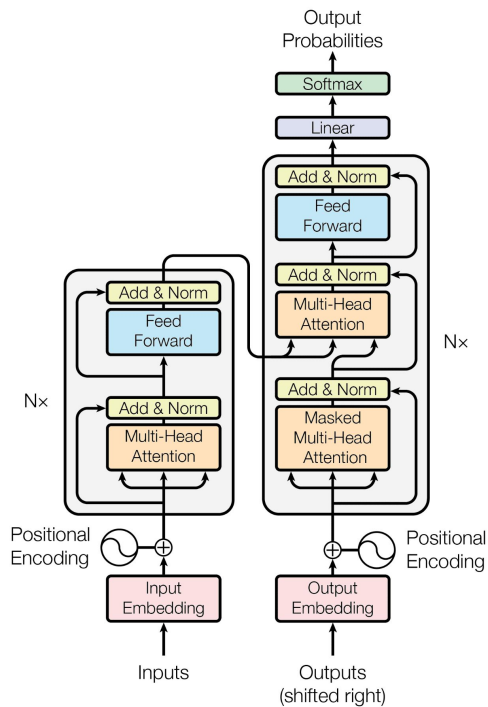


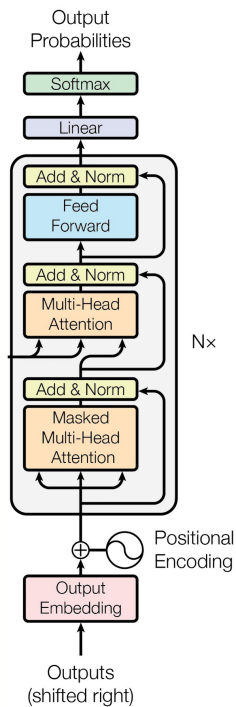
Follow-up topics:

- Other architecture choices (Dropout, Batchnorm, Layernorm,...)
- Impact of weight initialization

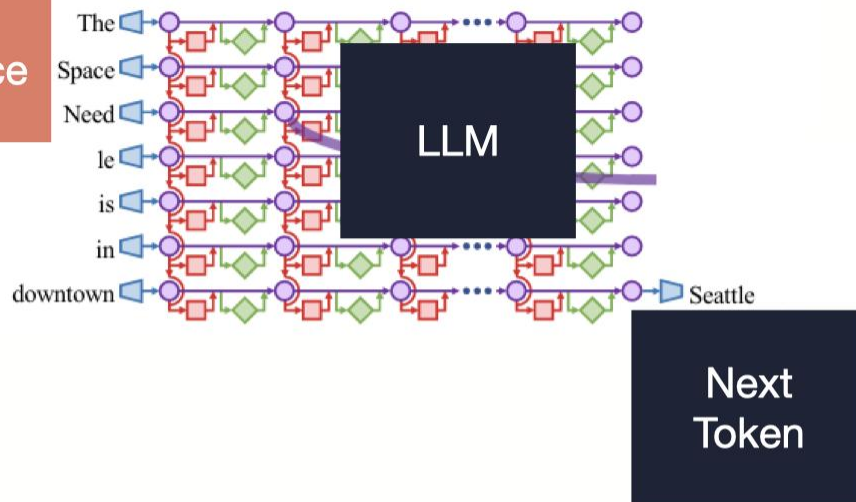


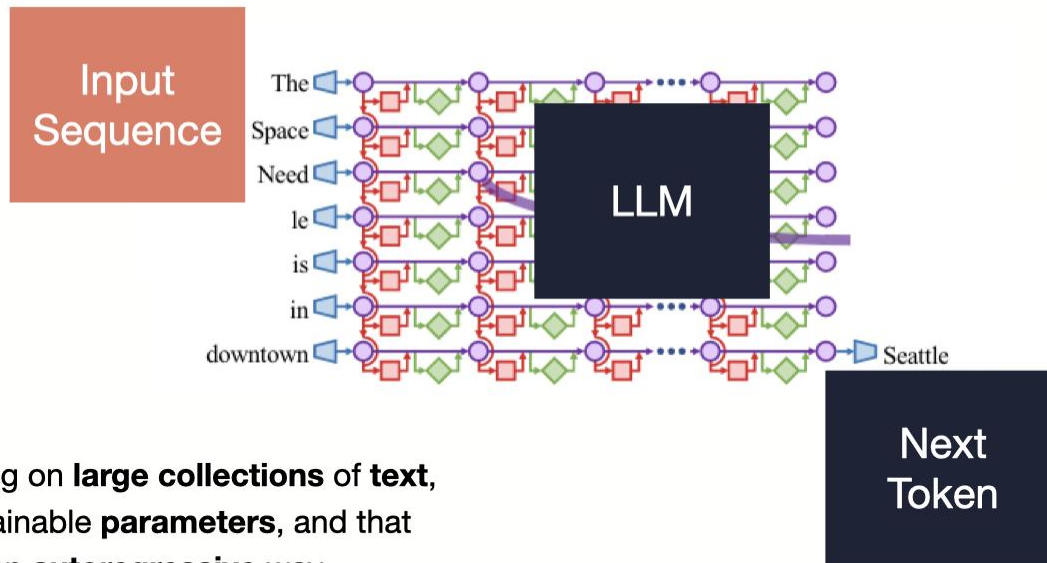
Transformer





Input
Sequence

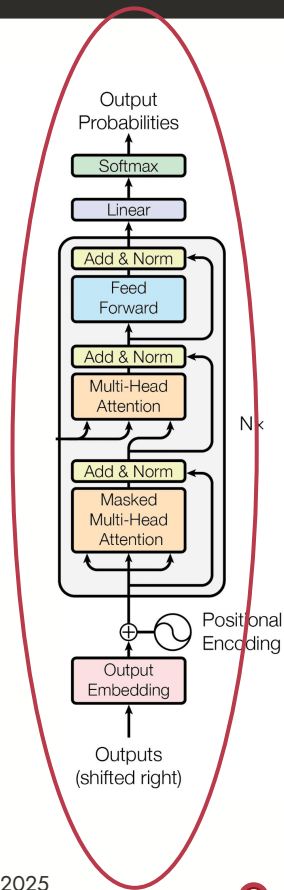




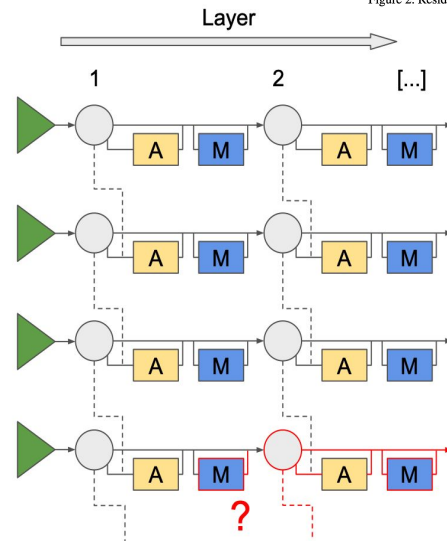
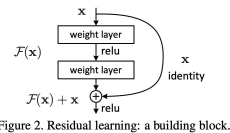
Created by training on **large collections of text**, with **billions** of trainable **parameters**, and that **generate** text in an **autoregressive** way.



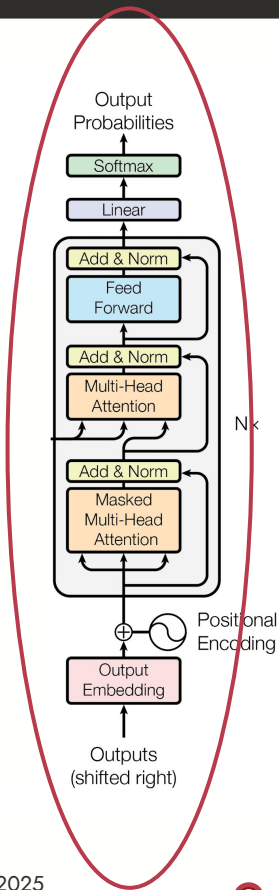
Transformer



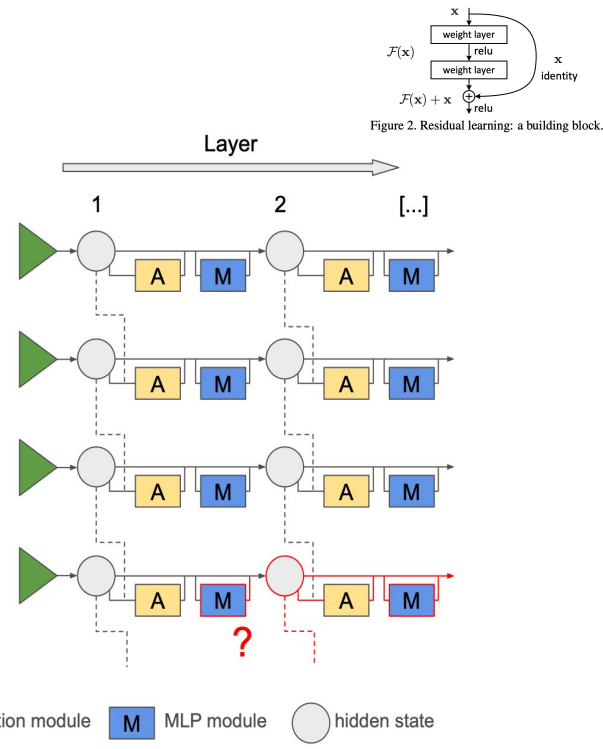
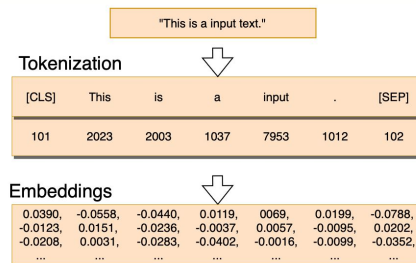
Decoder Only:



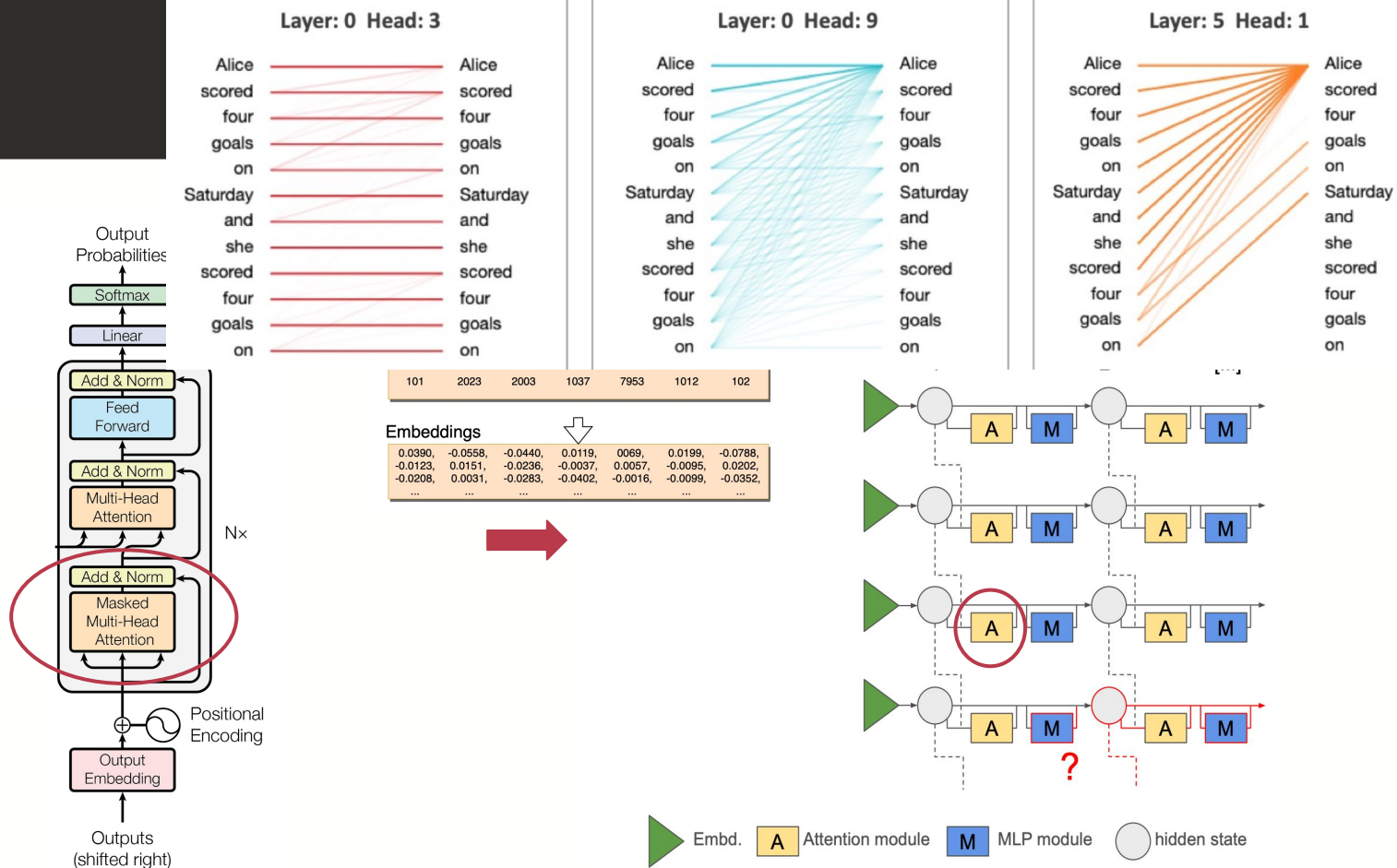
Transformer



Decoder Only:



Transformer



Vaswani, A. et al. Attention Is All You Need. Preprint at arxiv:1706.03762 (2017)

Vig, J. and Belinkov, Y. Analyzing the Structure of Attention in a Transformer Language Model. In Proceedings of the 2019 ACL Workshop BlackboxNLP (2019)

Me:
Question?

ChatGPT:
Answer!

- Great performance across variety of NLP tasks

Language Models are Unsupervised Multitask Learners

Alec Radford^{*1} Jeffrey Wu^{*1} Rewon Child¹ David Luan¹ Dario Amodei^{**1} Ilya Sutskever^{**1}

Abstract

Natural language processing tasks, such as question answering, machine translation, reading comprehension, and summarization, are typically approached with supervised learning on task-specific datasets. We demonstrate that language models begin to learn these tasks without any explicit supervision when trained on a new dataset of millions of webpages called WebText. When conditioned on a document plus questions, the answers generated by the language model reach 55 F1 on the CoQA dataset - matching or exceeding the performance of 3 out of 4 baseline systems without using the 127,000+ training examples. The capacity of the language model is essential to the success of zero-shot task transfer and increasing it improves performance in a log-linear fashion across tasks. Our largest model, GPT-2, is a 1.5B parameter Transformer that achieves state of the art results on 7 out of 8 tested language modeling datasets in a zero-shot setting but still underfits WebText. Samples from the model reflect these improvements and contain coherent paragraphs of text. These findings suggest a promising path towards building language processing systems which learn to perform tasks from their naturally occurring demonstrations.

competent generalists. We would like to move towards more general systems which can perform many tasks - eventually without the need to manually create and label a training dataset for each one.

The dominant approach to creating ML systems is to collect a dataset of training examples demonstrating correct behavior for a desired task, train a system to imitate these behaviors, and then test its performance on independent and identically distributed (IID) held-out examples. This has served well to make progress on narrow experts. But the often erratic behavior of captioning models (Lake et al., 2017), reading comprehension systems (Jia & Liang, 2017), and image classifiers (Alcorn et al., 2018) on the diversity and variety of possible inputs highlights some of the shortcomings of this approach.

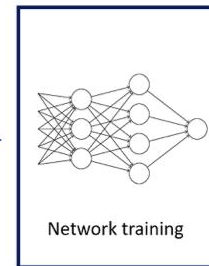
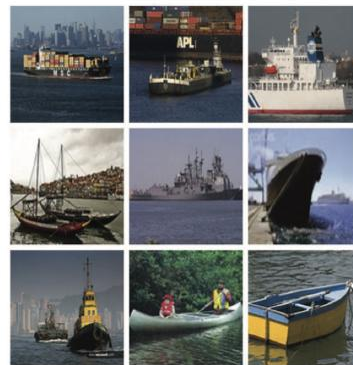
Our suspicion is that the prevalence of single task training on single domain datasets is a major contributor to the lack of generalization observed in current systems. Progress towards robust systems with current architectures is likely to require training and measuring performance on a wide range of domains and tasks. Recently, several benchmarks have been proposed such as GLUE (Wang et al., 2018) and decaNLP (McCann et al., 2018) to begin studying this.

Multitask learning (Caruana, 1997) is a promising framework for improving general performance. However, multitask training in NLP is still nascent. Recent work reports modest performance improvements (Yogatama et al., 2019) and the two most ambitious efforts to date have

Me:
Question?

ChatGPT:
Answer!

- Great performance across variety of NLP tasks



watercraft

Me:
Question?

ChatGPT:
Answer!

- Great performance across variety of NLP tasks

Language Models are Unsupervised Multitask Learners

Alec Radford^{*1} Jeffrey Wu^{*1} Rewon Child¹ David Luan¹ Dario Amodei^{**1} Ilya Sutskever^{**1}

Abstract

Natural language processing tasks, such as question answering, machine translation, reading comprehension, and summarization, are typically approached with supervised learning on task-specific datasets. We demonstrate that language models begin to learn these tasks without any explicit supervision when trained on a new dataset of millions of webpages called WebText. When conditioned on a document plus questions, the answers generated by the language model reach 55 F1 on the CoQA dataset - matching or exceeding the performance of 3 out of 4 baseline systems without using the 127,000+ training examples. The capacity of the language model is essential to the success of zero-shot task transfer and increasing it improves performance in a log-linear fashion across tasks. Our largest model, GPT-2, is a 1.5B parameter Transformer that achieves state of the art results on 7 out of 8 tested language modeling datasets in a zero-shot setting but still underfits WebText. Samples from the model reflect these improvements and contain coherent paragraphs of text. These findings suggest a promising path towards building language processing systems which learn to perform tasks from their naturally occurring demonstrations.

competent generalists. We would like to move towards more general systems which can perform many tasks – eventually without the need to manually create and label a training dataset for each one.

The dominant approach to creating ML systems is to collect a dataset of training examples demonstrating correct behavior for a desired task, train a system to imitate these behaviors, and then test its performance on independent and identically distributed (IID) held-out examples. This has served well to make progress on narrow experts. But the often erratic behavior of captioning models (Lake et al., 2017), reading comprehension systems (Jia & Liang, 2017), and image classifiers (Alcorn et al., 2018) on the diversity and variety of possible inputs highlights some of the shortcomings of this approach.

Our suspicion is that the prevalence of single task training on single domain datasets is a major contributor to the lack of generalization observed in current systems. Progress towards robust systems with current architectures is likely to require training and measuring performance on a wide range of domains and tasks. Recently, several benchmarks have been proposed such as GLUE (Wang et al., 2018) and decaNLP (McCann et al., 2018) to begin studying this.

Multitask learning (Caruana, 1997) is a promising framework for improving general performance. However, multitask training in NLP is still nascent. Recent work reports modest performance improvements (Yogatama et al., 2019) and the two most ambitious efforts to date have

Me:
Question?

ChatGPT:
Answer!

- Great performance across variety of NLP tasks
- Great zero/few-shot abilities on unseen tasks

Language Models are Unsupervised Multitask Learners

Alec Radford

Natural language understanding, text generation, and machine translation are all tasks that can be approached with specific datasets. Models begin to learn to perform these tasks by being trained on a large corpus of text. The capacity of the model to learn to perform these tasks is a function of the size of the corpus and the quality of the training data. The success of the model in these tasks is a function of the quality of the training data and the capacity of the model to learn from it.

Language Models are Few-Shot Learners

Tom B. Brown*	Benjamin Mann*	Nick Ryder*	Melanie Subbiah*
Jared Kaplan†	Prafulla Dhariwal	Arvind Neelakantan	Pranav Shyam
Girish Sastry	Amanda Askell	Sandhini Agarwal	Ariel Herbert-Voss
Gretchen Krueger	Tom Henighan	Rewon Child	Aditya Ramesh
Daniel M. Ziegler	Jeffrey Wu	Clemens Winter	
Christopher Hesse	Mark Chen	Eric Sigler	Mateusz Litwin
Benjamin Chess	Jack Clark	Christopher Berner	
Sam McCandlish	Alec Radford	Ilya Sutskever	Dario Amodei

Abstract

We demonstrate that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even becoming competitive with prior state-of-the-art fine-tuning approaches. Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3 achieves strong performance on many NLP datasets, including translation, question-answering, and cloze tasks. We also identify some datasets where GPT-3's few-shot learning still struggles, as well as some datasets where GPT-3 faces methodological issues related to training on large web corpora.

In-Context-Learning

During training, LLM develops a broad set of skills and pattern recognition abilities

outer loop

Learning via SGD during unsupervised pre-training

Can be used
During
inference

inner loop

1	5 + 8 = 13
2	7 + 2 = 9
3	1 + 0 = 1
4	3 + 4 = 7
5	5 + 9 = 14
6	9 + 8 = 17

↑
sequence #1

In-context learning

1	gaot => goat
2	sakne => snake
3	brid => bird
4	fsih => fish
5	dcuk => duck
6	cmihp => chimp

↑
sequence #2

In-context learning

1	thanks => merci
2	hello => bonjour
3	mint => menthe
4	wall => mur
5	otter => loutre
6	bread => pain

↑
sequence #3

In-context learning

Chain-of-Thought Reasoning

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✔️

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

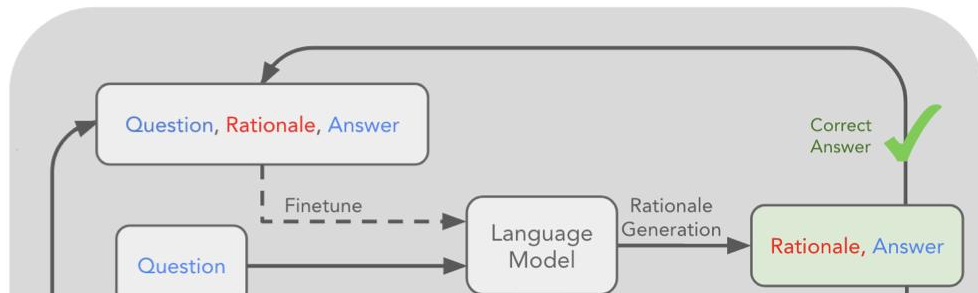
(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✔️*

Self-Taught Reasoning

Improving language model reasoning iteratively, building on pre-existing reasoning ability

1) Few-shot prompt a LM to self-generate rationales

2a) Fine-tuning on rationales with correct answers



Q: What can be used
to carry a small dog?
Answer Choices:
(a) swimming pool
(b) basket
(c) dog show
(d) backyard
(e) own home
A: The answer must be
something that can be
used to carry a small
dog. Baskets are
designed to hold things.
Therefore, the answer
is basket (b).

Self-Taught Reasoning

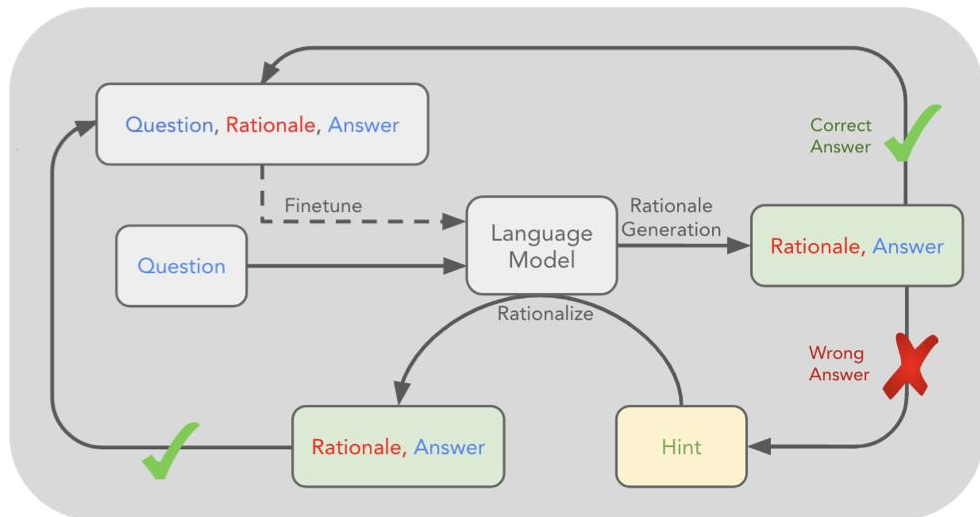
Improving language model reasoning iteratively, building on pre-existing reasoning ability

1) Few-shot prompt a LM to self-generate rationales

2a) Fine-tuning on rationales with correct answers

Or

2b) For wrong answers, generate a new rationale by providing the model with the correct answer
(backward reasoning, rationalization)



Q: What can be used to carry a small dog?

Answer Choices:

- (a) swimming pool
- (b) basket
- (c) dog show
- (d) backyard
- (e) own home

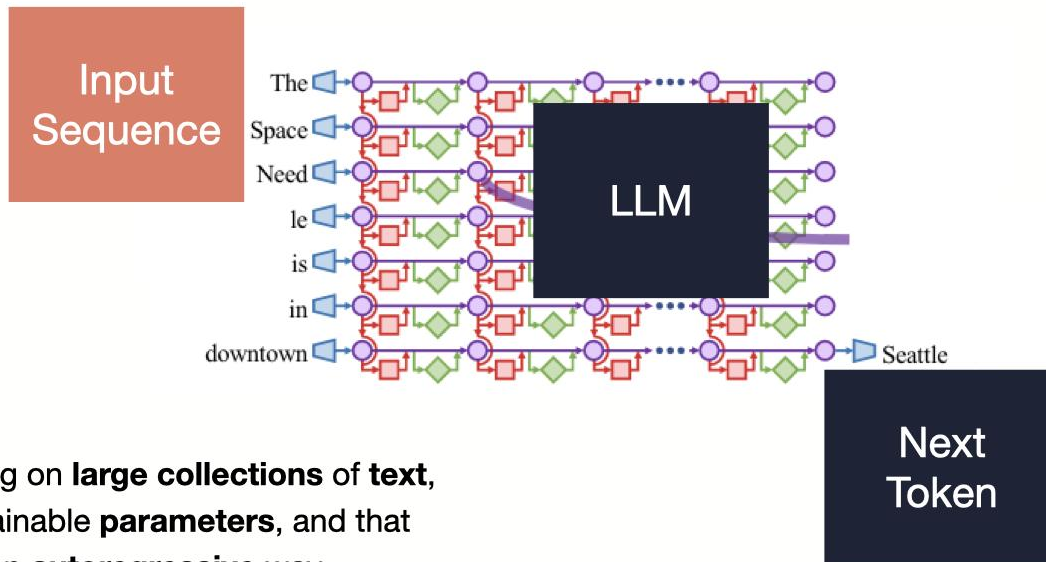
A: The answer must be something that can be used to carry a small dog. Baskets are designed to hold things. Therefore, the answer is basket (b).

Q: Where do you put your grapes just before checking out?

Answer Choices:

- (a) mouth
- (b) grocery cart (CORRECT)
- (c) super market
- (d) fruit basket
- (e) fruit market

A: The answer should be the place where grocery items are placed before checking out. Of the above choices, grocery cart makes the most sense for holding grocery items. Therefore, the answer is grocery cart (b).

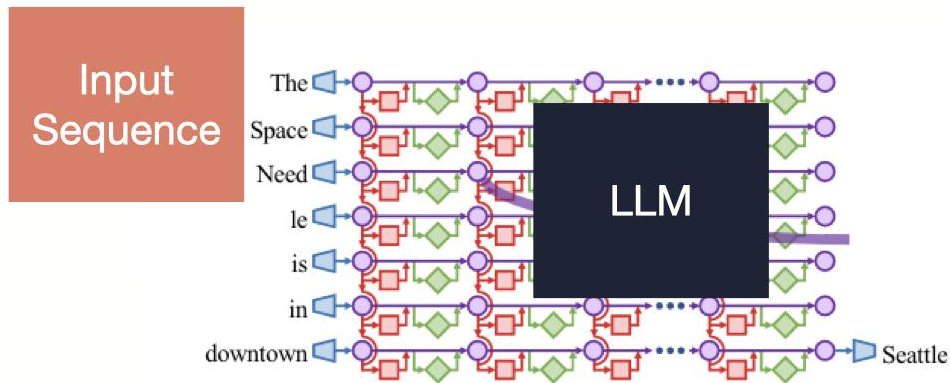


Created by training on **large collections of text**, with **billions** of trainable **parameters**, and that **generate** text in an **autoregressive** way.

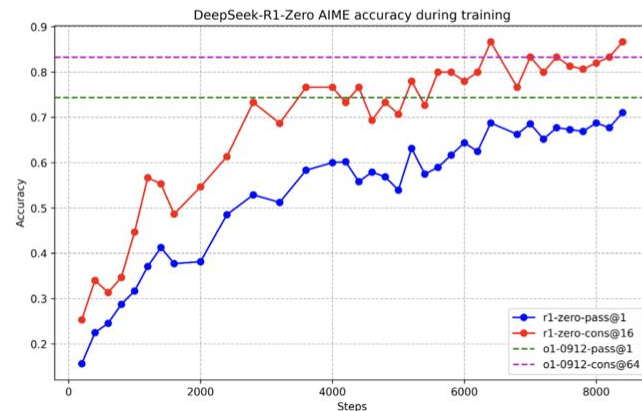


Image adapted from Meng, K., Bau, D., Andonian, A. & Belinkov, Y. Locating and Editing Factual Associations in GPT. NeurIPS (2022).

Image adapted from Raschka, S. New LLM Pre-training and Post-training Paradigms (2024)



Created by training on **large collections of text**, with **billions** of trainable **parameters**, and that **generate** text in an **autoregressive** way.



Modern **test-time compute scaling** (Reasoning-RL) to get models to **replicate** and **supersede scaffolding**

Image adapted from Meng, K., Bau, D., Andonian, A. & Belinkov, Y. Locating and Editing Factual Associations in GPT. NeurIPS (2022).

Image adapted from Raschka, S. New LLM Pre-training and Post-training Paradigms (2024)

DeepSeek-AI et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. Arxiv (2025)